

TECHNICAL PROJECT REPORT

PLC-linked machine vision for bottle label inspection

CODESYS | OPC UA | Python | OpenCV

Author	Noureddine Qarafi
School	ENSAM Meknès
Programme	Electromechanical Engineering
Project	Bottle label inspection
Academic year	2026 / 2027

Summary and contents

00 Project summary

I developed a laboratory prototype that links a simulated CODESYS PLC to a Python image-processing client. A bottle-detection event initiates an image check; a contour-based classifier looks for a label, then writes an ejection request to the PLC when it does not find the expected shape. The PLC logic and a simple HMI display the resulting state.

00.1 Objective

The goal is to demonstrate the complete signal path from a proximity event to an accept/reject indication. CODESYS Control Win V3 emulates the controller, two local bottle images stand in for camera frames, and OPC UA carries the process variables. The report documents the implemented sequence and the observations visible in the supplied screenshots.

00.2 Scope of the demonstration

Prototype boundary. The images are selected from local files. There is no physical camera, conveyor, bottle sensor or ejector in the demonstrated setup.

Contents

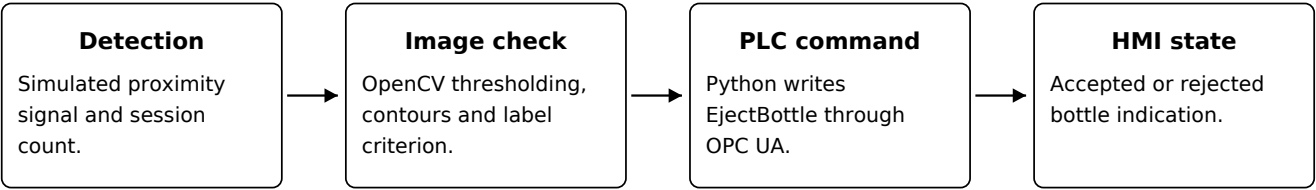
01	Architecture and communication	3
02	PLC sequence and ejection control	4
03	Sessions and status indicators	5
04	Python vision and decision logic	6
05	HMI and observed tests	7
06	Limits and conclusion	8

Architecture and communication

01 System roles and exchanged variables

CODESYS Control Win V3 x64 hosts the emulated PLC and exposes application variables through its OPC UA server. The Python process is the client. UaExpert is used independently to inspect the exposed nodes and their current values.

1.1 Signal path



1.2 Exchanged signals

- Bottledetected and sessinNumber.** The PLC reports a detection event and a session identifier to Python.
- EjectBottle.** Python writes the rejection request; the PLC clears it after its eject timer.
- exit_script.** The PLC exposes an operator stop condition to the client loop.
- accepted and unaccepted.** PLC Boolean expressions drive the HMI indications.

1.3 OPC UA link

The emulated CODESYS PLC is the OPC UA server and Python is the client. The default endpoint is **opc.tcp://DESKTOP-QUBUSE2:4840**. UaExpert separately showed the application tags with Good status during the captured session.

#	SERVER	NODE ID	NODE PATH	DISPLAY NAME	VALUE	D
1	OPCUAServer@DE...	NS4 String var C...	Programs/...edetecte	Bottledetected	true	Boolea
2	OPCUAServer@DE...	NS4 String var C...	Programs/...ectBottle	EjectBottle	false	Boolea
3	OPCUAServer@DE...	NS4 String var C...	Programs/.../Proximity	Proximity	false	Boolea
4	OPCUAServer@DE...	NS4 String var C...	Programs/...ect_relay	eject_relay	true	Boolea
5	OPCUAServer@DE...	NS4 String var C...	Programs/P...xit_script	exit_script	false	Boolea
6	OPCUAServer@DE...	NS2 Numeric 6095	DeviceTop...ineAccess	OnlineAccess		Null

Figure 1. Enlarged monochrome crop of the UaExpert data access view, showing the exposed PLC tags and values.

PLC sequence and ejection control

02 Bottle event and timed ejection

The ladder program generates a simulated proximity pulse, latches a bottle-detected event at its rising edge, and resets that flag after a timed interval. A Python rejection writes EjectBottle, which drives the eject relay flag until the PLC timer resets the command and detection state.

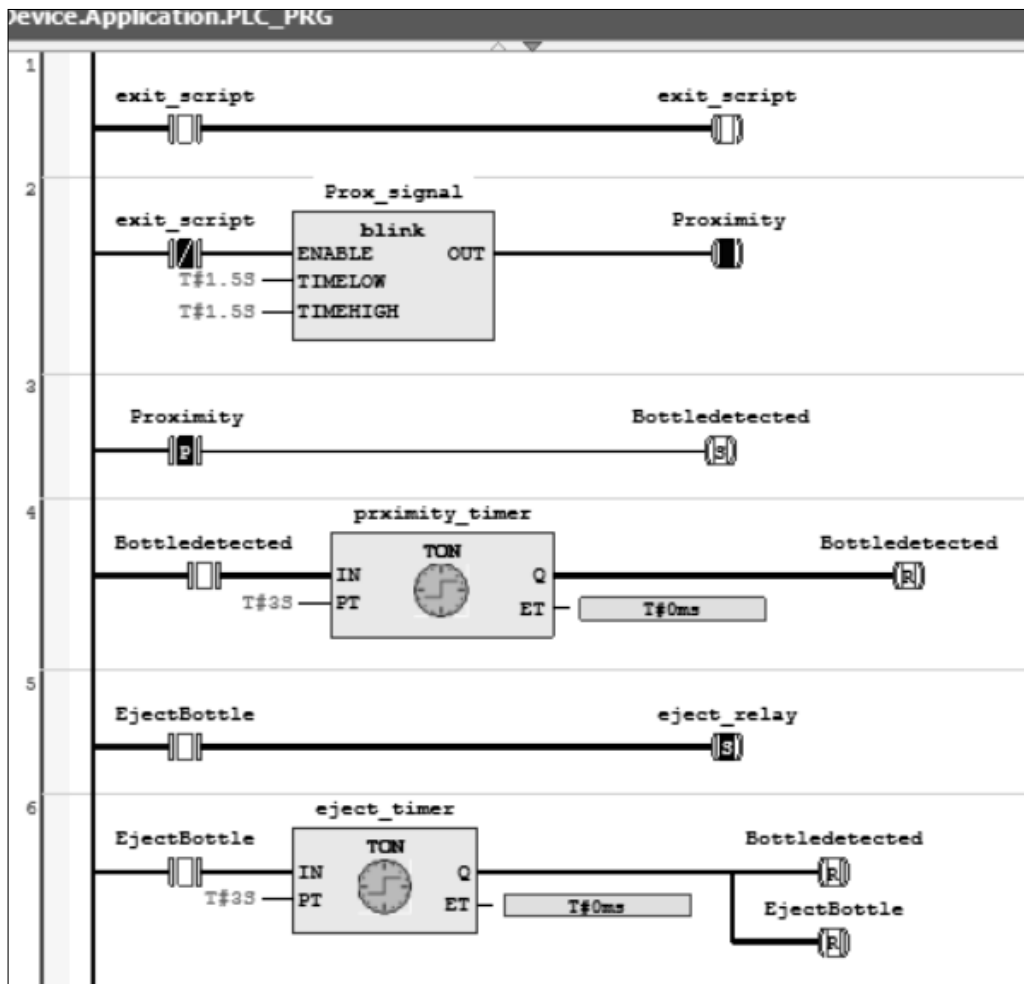


Figure 2. Main CODESYS ladder networks: proximity simulation, event latch, 3 s event timer, eject relay and 3 s eject reset (supplied screenshot).

2.1 Detection

A Blink block simulates Proximity. Its rising edge sets Bottledetected, which is reset by proximity_timer after a nominal T#3s.

2.2 Ejection

When Python writes EjectBottle, the PLC sets eject_relay. eject_timer then clears the detection and ejection commands after a nominal T#3s.

2.3 Implementation scope

The sensor and relay are software signals. Their timing has not been validated against physical conveyor movement.

Session handling and status indication

03 One classification per detected session

A count-up block increments when Bottledetected becomes active, resets at a preset of five, and transfers its current value into the program variable spelled **sessinNumber**. Python reads this variable together with Bottledetected and compares it with a local last_session value before processing another image.

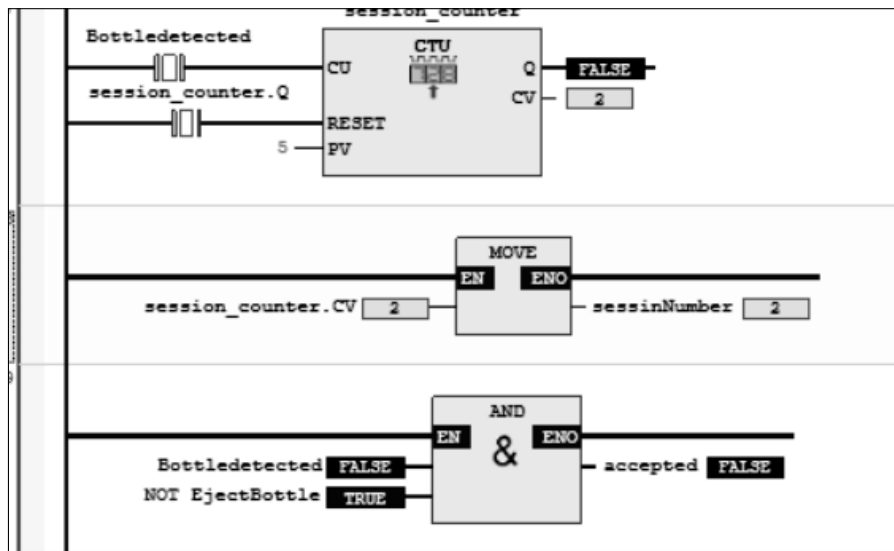


Figure 3. CODESYS session counter, MOVE to sessinNumber and initial status networks (supplied screenshot).

3.1 Synchronization with the client

The Python loop checks both Bottledetected and a changed session value before it selects the next image. The variable is spelled sessinNumber in the implemented code and PLC screen.

3.2 Accepted and rejected display flags



Figure 4. Status logic shown online: accepted = Bottledetected AND NOT EjectBottle; unaccepted = Bottledetected AND EjectBottle. The capture shows the rejected condition active.

These flags describe the instantaneous PLC state. They do not record a permanent quality result after the detection timer has reset. A production counter or trace would require separate retained result variables.

Python image-processing method

04 Image-processing sequence

The Python client selects one of two local images, reads it in grayscale with OpenCV and applies adaptive mean thresholding. Contour extraction and polygon approximation then form a simple geometric label criterion. The same selected file is read in color so a matching contour can be drawn for visual inspection.

4.1 Image acquisition

A random choice between two local JPEG files simulates frame acquisition. The selected file is loaded once in grayscale and once in color for contour drawing.

4.2 Thresholding

OpenCV adaptive mean thresholding converts the grayscale frame to a binary image using an 11-pixel block and $C = 5$.

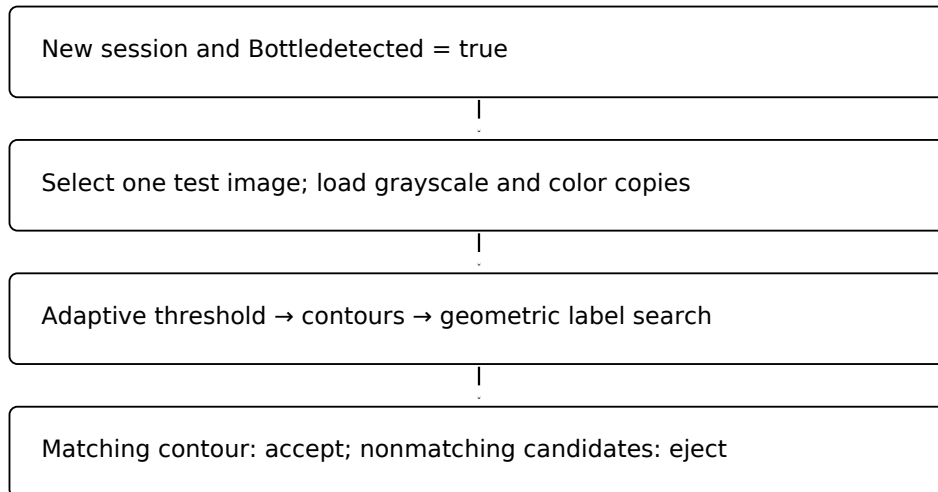
4.3 Contour search

findContours uses RETR_TREE and CHAIN_APPROX_NONE. Each candidate is measured by area and perimeter, then approximated by approxPolyDP with $\epsilon = 0.02 \times \text{perimeter}$.

4.4 Label criterion

A label candidate has four vertices and an area strictly between 142 and 31320 pixels in the current code.

4.5 Decision path



The classifier returns an ejection decision only after no contour satisfies the label test. On rejection, the client writes the Boolean EjectBottle tag through OPC UA and increments a local rejected-piece count.

Current code edge case: when findContours returns no contours, check_label starts with should_eject = False and returns False. An empty contour set is therefore accepted by the current implementation; the default should be changed and tested before relying on it.

HMI and observed tests

05 Operator view

The CODESYS HMI groups the exit control, simulated proximity and ejection controls with red and green bottle indications. It provides a compact way to observe the PLC variables while the Python process performs image classification.

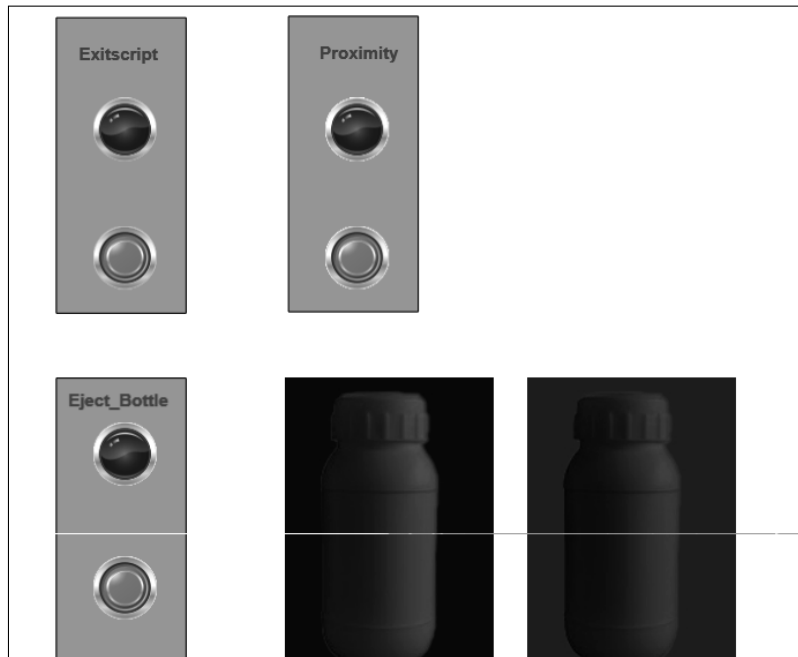


Figure 5. Monochrome HMI crop. The left bottle panel is the reject indication; the right panel is the accept indication.

5.1 Evidence from the recorded session

```
-----
EjectBottle activated
3
-----
no label detected . activate Ejection
-----
EjectBottle activated
4
█
```

Figure 6. Python terminal extract from the same capture: “no label detected”, EjectBottle activated and a rejected-piece count reaching four.

Communication. UaExpert showed readable PLC nodes with Good status in the captured session.

Reject case. The Python console printed no-label and EjectBottle-activated messages; the PLC capture showed unaccepted = true.

Accept case. The accept expression and HMI indication are present, but a separate measured pass-rate test was not supplied.

Limits and conclusion

06 Engineering limits and next tests

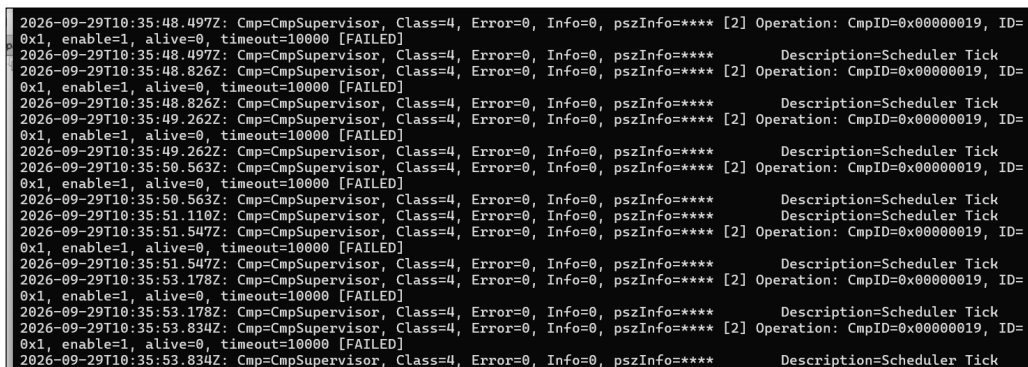
01. Image source: random selection between two fixed file paths replaces a triggered camera capture; there is no calibration of lens, lighting or conveyor position.

02. Classification: contour area and four-corner thresholds are hand-set for the example images. False accept/reject rates and robustness to rotation, glare and occlusion have not been measured.

03. Synchronization: Python uses a locally advanced last_session counter rather than assigning the actual PLC sessionNumber value. A mismatch after a reset could skip or repeat a bottle.

04. Recovery: the client loop has no reconnection, image-load check, explicit timeout policy or delay; these need implementation for sustained operation.

6.1 Runtime diagnostic



```

2026-09-29T10:35:48.497Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:48.497Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:48.826Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:48.826Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:49.262Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:49.262Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:50.563Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:50.563Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:51.110Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:51.547Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:51.547Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:53.178Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:53.178Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick
2026-09-29T10:35:53.834Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** [2] Operation: CmpID=0x00000019, ID=
0x1, enable=1, alive=0, timeout=10000 [FAILED]
2026-09-29T10:35:53.834Z: Cmp=CmpSupervisor, Class=4, Error=0, Info=0, pszInfo=**** Description=Scheduler Tick

```

Figure 7. A later CODESYS Control Win V3 console capture contains repeated Scheduler Tick / [FAILED] messages (supplied screenshot).

This log does not erase the observed OPC UA exchange and reject sequence, but it prevents a claim of stable continuous operation. A repeat test should correlate PLC runtime logs, session numbers, Python decisions and HMI results over a defined batch.

6.2 Conclusion

I integrated a CODESYS PLC emulator, OPC UA client, Python/OpenCV classifier and HMI into a working demonstration of bottle-label rejection. The supplied captures show accessible PLC tags and an executed rejection command. The prototype is useful for learning the interface between machine vision and automation; the next step is to correct the identified edge cases and validate it with a controlled image set and a reliable runtime.

6.3 Learning reference

Mouhammad Hamsho and Kemalaldin Hamso, PLC Basic Machine Vision | From Scratch, Udemy (course page, accessed 29 September 2026): <https://www.udemy.com/course/plc-basic-machine-vision-from-scratch/>

Project evidence: plc_connecting_code.py and the CODESYS, UaExpert, HMI and runtime screenshots supplied with this report. Figures are the author's project captures.